

xxxx 实验报告

姓名： 林宏宇 学号： 23320093

1. 实验内容

本次实验使用 Vivado 的 Block Memory Generator 模拟数据在存储器中的存取过程。实验使用单端口 ROM。初始化 ROM 存储器中的内容，通过串口的 RX 接收地址，将对应的存储器中内容读出来，并通过 TX 发送回 PC。实验原理如图 1 所示：

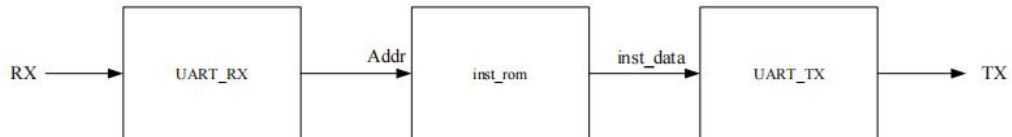


图 1 存储器实验原理图

2. 实验过程

1.生成 ip 核



2.创建顶层模块



3. 在 top_module 里实现逻辑，控制握手信号

例化部分

```
module top_module (  
    input wire clk,           // System clock  
    input wire rst_n,        // Active-low reset  
    input wire rx,           // UART receiver input  
    output wire tx           // UART transmitter output  
);  
  
// Parameters  
parameter BAUDRATE = 115200;  
  
// Signals  
wire rx_valid;              // Data valid signal from RX  
wire [7:0] rx_data;         // Received data  
wire tx_ready;              // Ready to transmit  
wire tx_valid;              // Valid signal for TX  
reg [31:0] tx_data;         // Data to transmit  
reg rx_ready;               // Ready signal for RX  
  
// Instantiate UART top module  
uart_top #(  
    .baudrate(BAUDRATE)  
) u_uart_top (  
    .clk_i(clk),  
    .rst_n(rst_n),  
    .rx_i(rx),  
    .tx_o(tx),  
    .tx_valid_i(tx_valid),  
    .tx_ready_o(tx_ready),  
    .tx_data_i(tx_data),  
    .rx_valid_o(rx_valid),  
    .rx_ready_i(rx_ready),  
    .rx_data_i(rx_data)  
);  
  
// ROM output  
wire [31:0] rom_data;  
reg [7:0] rom_addr;  
  
// ROM instance  
inst_rom u_inst_rom (  
    .clka(clk),  
    .ena(rom_enable),  
    .addra(rom_addr),  
    .douta(rom_data)  
);
```

使用状态机控制握手信号

```

// State encoding using parameters
localparam [1:0] IDLE      = 2'b00,
               SEND_DATA = 2'b01;

// State register
reg [1:0] state;

// Single always block for state transition and outputs
always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        state <= IDLE;
        rom_addr <= 8'd0;
        tx_data <= 32'd0;
        rx_ready <= 1'b0;
    end else begin
        case (state)
            IDLE: begin
                rx_ready <= 1'b1; // Prepare to receive data
                if (rx_valid) begin
                    rom_addr <= rx_data; // Capture address
                    rx_ready <= 1'b0; // Disable RX
                    state <= SEND_DATA; // Transition to SEND_DATA
                end
            end
            SEND_DATA: begin
                if (tx_ready) begin
                    tx_data <= rom_data; // Load ROM data for TX
                    state <= IDLE; // Return to IDLE
                end
            end
        endcase
    end
end

// Output assignments
assign rom_enable = (state == SEND_DATA);
assign tx_valid = (state == SEND_DATA);

endmodule

```

4. 结果分析

分别输入地址 01、02、03、04 可以得到在存储器上对应的地址，

结果正确，验收通过，代码正确

```
memory initialization radix=1
memory initialization vector=
00000000,
00000108,
23320093,
00000304,
00000506,
00000907,
00000001,
00000004,
00000005,
00000009,
00000007,
00000005,
00000000,
00000005,
00008888,
00008888,
00009999,
00009999,
0000Aaaa,
0000zzzz
```

ATX 正点原子串口调试助手 V2.8

[2024-11-22 22:12:14.136]

TX: 01

[2024-11-22 22:12:15.146]

RX: 00 00 01 08

[2024-11-22 22:12:21.938]

TX: 02

[2024-11-22 22:12:22.962]

RX: 23 32 00 93

[2024-11-22 22:12:27.666]

TX: 03

[2024-11-22 22:12:28.682]

RX: 00 00 03 04

[2024-11-22 22:12:32.605]

TX: 04

[2024-11-22 22:12:33.632]

RX: 00 00 05 06

[2024-11-22 22:12:45.241]

TX: 05

[2024-11-22 22:12:46.250]

RX: 00 00 09 07

单条发送 多条发送 协议传输 帮助

05]

5. 总结

- 1.跟着实验课 ppt 操作时，刚开始不熟悉流程，不知道应该建立什么模块，顶层模块还是分支模块，不知道如何在顶层模块里引用 uart，不知道如何正确地写代码
- 2.经过几天的钻研学习，逐渐明白了应该如何操作，创建 ip 核、读懂代码、创建模块例化、利用握手信号写代码、烧录、串口调试。
- 3.对串口协议有了更深入的了解，对代码的封装和引用也有了新的学习认识